

12. AI 활용 내역 요약

12-1. 팀 전체 AI 활용 현황

항목	내용
총 활용 횟수 (추정)	약 120회
주요 사용 도구	ChatGPT (GPT-4o), Gemini
가장 많이 활용한 단계	구현·검토 (M3)

12-2. 단계별 활용 내역

단계	주요 활용 내용	활용 도구	팀 수정 여부
기획 (M1)	타겟 사용자(대학생 및 일반 컴퓨터 비전문가)의 니즈 구체화를 위한 가상 페르소나 설계 및 업그레이드 시 잠재적 대기 불편 사항 도출	ChatGPT	수정 후 반영
설계 (M2)	단방향 연관 관계 구조 모델링 검토 및 Facade/Controller 패턴의 적절성에 대한 소프트웨어 아키텍처 피드백 확보	ChatGPT	수정 후 반영
구현·검토 (M3)	- 외부 라이브러리 없이 구동하는 동적 SVG 레이더 차트 좌표 계산식 생성 - 가격 크롤러 차단 방지용 requests 가짜 헤더 구성 및	ChatGPT, Gemini	수정 후 반영

	RAM 용량 파싱 정규표현식 보완		
--	-----------------------	--	--

12-3. AI 활용 3원칙 준수 자체 평가

원칙	준수 여부	비고
단순 복사 금지	준수	AI가 최초 제안한 복잡한 빌더(Builder) 설계 패턴 대신, 우리 프로젝트 규모에 어울리는 최적의 단일 상태 관리 구조(state 객체) 및 간결한 Facade(App 객체) 패턴으로 개발진이 리팩토링하여 적용함.
비판적 검증	준수	크롤링 실패 시 AI가 제시한 단순 터미널 에러 로깅 코드가 프론트엔드의 비정상 종료(Crash)를 야기함을 식별하고, 비동기 예외 시 더미 안전 데이터(Mock Data)를 대체 공급하는 무중단 보완 코드를 설계함.
수정 이력 명시	준수	리플(Ripple) 물결 애니메이션 함수 및 동적 차트 연산부(drawRadar) 등 인공지능이 제공한 특정 좌표 수학적 블록 영역 상단에 // AI-generated 주석 가이드라인을 철저하게 명시함.

12-4. 가장 효과적이었던 AI 활용 사례

- 대상 작업: 외부 라이브러리를 쓰지 않는 초경량 동적 SVG 레이더 차트(drawRadar) 좌표 연산식 개발
- 활용 방법:
 - 사용자의 진단 점수 데이터(CPU, GPU, RAM, 저장장치, 종합 점수)를 바탕으로

오각형의 정점 좌표를 동적으로 그려내기 위해 삼각함수 공식을 활용한 다각형 생성 수식을 질문했습니다.

- AI가 계산해 준 중심점 $\$cx, cy\$$ 대비 각 축의 라디안 점 도출 수식 공식인 $\$cx + d \cdot \cos(\theta)\$$ 및 $\$cy + d \cdot \sin(\theta)\$$ 변환 함수 코드를 바닐라 자바스크립트로 변환하여 App.drawRadar 내부 SVG 폴리곤 템플릿 리터럴로 직접 조립해 냈습니다.
- 수행 결과:
 - 고용량의 외부 차트 엔진(Chart.js 등)을 일절 로드할 필요가 없어 일렉트론 앱의 최초 구동 및 패키징 용량을 획기적으로 경량화했습니다.
 - 동시에 렌더링 리소스를 최소화하여 M3 최종 검증 단계에서 100% 무결하게 일치하는 반응형 결과 차트 뷰를 단 30라인의 스크립트만으로 완성해 냈습니다.

12-5. AI 활용의 한계 또는 주의가 필요했던 사례

- 대상 작업: 실시간 다나와 최저가 웹 크롤러 엔진(PriceSearcher) 셀렉터 파싱 및 자동화 개발
- 한계 극복 과정 및 교훈:
 - AI에게 타겟 하드웨어 쇼핑물의 HTML 원시 구조를 제공하고 최적의 크롤링 코드 작성을 요청했으나, AI는 대상 웹페이지가 자바스크립트에 의해 실시간 동적 로딩되는 동적 돔(DOM) 구조임을 완전히 간과한 채 Beautiful Soup의 단순한 정적 HTML 파서 코드를 제안했습니다. 이로 인해 최초 빌드 시 무조건 가격 필드가 None으로 반환되는 로직 결함이 드러났습니다.
 - AI의 제안을 무비판적으로 복사·붙여넣기 했다면 크롤러가 영구 동작하지 않았을 것입니다.
 - 개발팀은 직접 크롬 개발자 도구를 활용해 실시간 네트워크 비동기 API 요청 경로를 재분석하였고, 수집 시점에 가짜 브라우저 헤더(User-Agent)를 심지 않으면 타겟 서버에서 봇 트래픽으로 오인해 접근을 차단한다는 네트워크 보안 예외 사항을 직접 보완 구현했습니다. 이 과정을 통해 인공지능이 제시하는 코드는 반드시 개발팀의 실시간 도메인 디버깅과 검증을 필히 거쳐야만 상용 소프트웨어에 이식될 수 있다는 점을 뼈저리게 배웠습니다.